

Project 3 – Sprint 4 Retrospective Change Plan

You must complete all sections clearly and honestly.

Section 1 – Sprint Outcome

- Sprint Goal (copy exactly from your Sprint 4 Plan):
By the end of Sprint 4, a user will be able to upload both images AND documents to their personal galleries; calculate the CO2 savings of choosing to bike instead of driving, using public transit, or walking for a given distance; and log a bike ride to a personal activity log that persists.
- Sprint Goal Met:
Yes
- In ONE sentence, what can a user now do?
(Must match your demo – no vague descriptions)
A user can now upload both images and documents to their personal galleries, view the CO2 emissions of biking, walking, public transport, and driving when creating a route, and log a bike activity to a personal activity log that persists.
- During your demo, did you clearly show ALL parts of your Sprint Goal without needing follow-up questions?
Partially - some parts required clarification during Q&A
- What parts of your Sprint Goal were NOT clearly demonstrated during the initial demo?
(Include anything that had to be explained, skipped, or shown only after prompting)
We did not explicitly demonstrate the gallery being unique for every user, and only after some prompting did we show that it was different for each user.
- Why were these parts not demonstrated clearly?
(Choose the closest cause)
Did not realize it was required to show
Explain:
Having individual personal galleries was a feature of our last Sprint Goal and we demonstrated the feature during our last Sprint (Sprint 3), so we did not realize that we needed to show the feature again during our current Sprint.
- Did your system work consistently during the demo?
Yes
- What specifically failed, broke, or behaved inconsistently?
(Be concrete – errors, crashes, missing persistence, etc.)
The only thing that could be considered an error was that one of our routes did not generate. However, the only reason this occurred is because we placed a route on a waterway. Other than that, everything was demonstrated as expected. This was not a system failure, rather a lack of pre-determined route.

Section 2 – Structural Failure (Evidence Required)

- Primary Structural Issue (choose one):
Testing too late
- Describe exactly what happened:

Project 3 – Sprint 4 Retrospective Change Plan

Every Sprint Goal component met its task-level Definition of Done, but the full set of features was never exercised as one continuous demo before demo day. Individual tests confirmed each task worked however nothing tested them together. Integration friction, persistence verification time, and visual ambiguity in the CO2 chart came up during the demo.

- Evidence 1 (REQUIRED – must be observable):

Demo behavior

Explain:

During the demo, three moments required follow-up Q&A: activity log persistence was only confirmed after a manual refresh prompted by an audience question, the CO2 comparison bar scaling required verbal explanation, and form state clearing between page transitions was not anticipated. None were caught before the demo.

Evidence 2 (REQUIRED – different from above):

GitHub (commits / PR / timing)

Explain:

GitHub history shows our final commits being merged to the main branch the day before our Demo, and we had zero commits or PRs related to testing or end-to-end walkthroughs.

Section 3 – Root Cause (System-Level Only)

- Complete this sentence:

This happened because our system allowed:

Individual tasks to be marked as Done and tested/verified in isolation without ever requiring all components of the Sprint Goal to be tested thoroughly end-to-end in a fresh session.

- Explain the system failure clearly:

(No blaming people. Focus on structure, timing, workflow, and enforcement gaps.)

Our team's Definition of Done requires tasks to be tested individually, and our Sprint 3 Rule only ensures that every Sprint Goal component has a task. Both of these worked and helped our system succeed. Our system failure originated from the fact that there is nothing in our workflow that requires all completed features to run together as a continuous flow during development days. Testing mainly occurred when tasks were completed, but it was only ever per-task. There was no task where one person from our team runs through and tests every complete feature. Our demo was the first such run, which is why it was a little bit rough.

Section 4 – Sprint 4 Rule Evaluation (From Sprint 3 Retrospective Change Plan)

- Sprint 3 Retrospective Rule (copy exact wording from your Sprint 3 Change Plan)
At the start of every development day, the Scrum Master (Aarush) will verify that every component of the Sprint Goal has at least one corresponding task on the GitHub Sprint 4 Board with an assigned owner before the team begins ANY development work.

Project 3 – Sprint 4 Retrospective Change Plan

- Was this rule followed consistently during Sprint 4?

Always

- Who was responsible for enforcing this rule?

Name + Role: **Aarush + Scrum Master**

- Did this person actually enforce it during Sprint 4?

Yes

Break Point Analysis (REQUIRED)

- When did this rule fail during the Sprint?

(Be specific – day, situation, or workflow moment)

It didn't. Aarush ran the goal-coverage check at the start of every dev day, and every Sprint Goal component had a task with a named owner from Day 1. Sprint 4 is the first sprint we fully met the Sprint Goal. The failure that surfaced (testing too late) is downstream of what this rule was designed to address.

- What should have happened instead?

Nothing about the Sprint 3 rule's enforcement should change. It succeeded at its job as we finished all our work. What's missing is a complementary rule that would have ensured that an additional failure net was created. We needed to validate that completed features run together cleanly, not just that each one is tracked. Sprint 5 adds that rule alongside this one, not only as a replacement but one that can work together with.

- What evidence shows whether this rule was followed or not?

(Must reference demo, GitHub, or board behavior)

Sprint 4 Project Board on Dev Day 1 shows all three Sprint Goal components (document upload, CO2 calculator, activity log) with task cards and named owners, confirming Aarush ensured that everything in our sprint was completed and assigned. Zero Sprint Goal components went untracked which is huge progress from Sprint 3, where document upload had no board activity. This time, everything on the project board was completed and marked "Done"!

Section 5 – Refined Rule for Sprint 5 (STRICT FORMAT REQUIRED)

- You MUST use this format:

At [time], [role] will [observable action].

Your Rule:

At the final 10 minutes of every class period beginning Dev Day 3, Yohaán (Demo Verifier) will run an unbroken end-to-end walkthrough of every completed Sprint Goal component on a clean, logged-out browser, and any feature that fails, requires verbal explanation, or behaves inconsistently must be moved back to "In Progress" on the GitHub board before class ends.

- What specific problem does this rule solve?

(Must directly connect to Section 2 failure)

Directly addresses the Section 2 failure (testing too late). Forces a recurring full-system walkthrough during dev days, so integration friction, UX gaps, and persistence ambiguity

surface while there's still time to fix them, not at the demo. Targets the gap between "each task works" and "the full Sprint Goal demos cleanly."

Section 6 – Enforcement System (NON-OPTIONAL DETAIL)

- WHO is responsible for enforcement?
Name + Role: **Sathkrith**
- WHEN will this be enforced?
(Specific moment – not "throughout")
This will be enforced 5 minutes after class every day. That way all of us have a designated time we can always meet up.
- HOW will this be checked during daily work?
(What will someone SEE happening?)
The team sees Yohaán project his screen, open a fresh incognito tab, log in as a test account, and run through every completed Sprint Goal component back-to-back without pausing (upload a file, run a CO2 calc, log a ride, open the calendar, etc). Any feature that requires verbal explanation, fails, or behaves inconsistently is flagged on the GitHub board in real time during the walkthrough.
- What happens immediately if this rule is NOT followed?
(Must define an action – not "we will remind them")
Every task moved to Done since the last completed walkthrough is automatically reverted to "In Progress" on the GitHub board. No new task may move to Done at the next class until the missed walkthrough is run.
- Failure Scenario (REQUIRED):
If this rule is ignored for one full class period, what happens next?
If skipped for one full class, the next class begins with a mandatory two-walkthrough catch-up, one for the missed day's completed features, one for the current day's, and no development work starts until both pass. If skipped two classes in a row, Aarush (Scrum Master) removes the most recently added Sprint Goal component from scope to shrink the demo surface.

Section 7 – Measurable Indicator of Success

These must be observable and verifiable (board, GitHub, or demo).

- Indicator 1 (REQUIRED):
Every sprint goal component from the Sprint 4 plan has a corresponding task card on the GitHub board and is assigned to someone by the end of the first day of this sprint. This is verifiable by comparing the sprint goal list to the board at any point during the sprint.
- Indicator 2 (RECOMMENDED):
No sprint goal feature is missing from the demo because it lacked a task. At demo time, every stated goal component either has a Done card on the board.

Section 8 – Rule Validation Checklist (REQUIRED)

Your rule is only valid if ALL boxes can be checked:

- ☒ This rule can be observed during class

Project 3 – Sprint 4 Retrospective Change Plan

- ☒ This rule has a specific owner
- ☒ This rule happens at a clearly defined time
- ☒ This rule would be visibly broken if not followed
- ☒ This rule directly prevents this Sprint's failure
- ☒ This rule is enforced before demo day

If any box cannot be checked, your rule is not valid and must be revised.